NEXOVOLT

# Informe técnico de NexoVolt

Fundamentos físicos, arquitectura, algoritmos, pruebas y validación

**Aplicación educativa de electromagnetismo**

React, TypeScript, Vite, Tauri y Rust

Versión de entrega 10 de noviembre de 2026

Repositorio NexoVolt

# Resumen ejecutivo

NexoVolt es una aplicación educativa de escritorio para simular fenómenos de electromagnetismo mediante cuatro estaciones interactivas. El frontend está construido con React, TypeScript y Vite; Tauri 2 proporciona el contenedor de escritorio y un backend Rust para los cálculos físicos verificables. La interfaz también conserva implementaciones equivalentes en TypeScript para funcionar durante el desarrollo web.

La revisión del proyecto confirmó una compilación de producción correcta, la ejecución satisfactoria de quince pruebas unitarias en Rust y la carga funcional de las cuatro estaciones. Las simulaciones combinan modelos analíticos, integración Runge-Kutta de cuarto orden y visualizaciones SVG/CSS. El alcance es educativo: las geometrías y materiales se modelan mediante aproximaciones ideales.

# Objetivo y alcance

El producto permite manipular parámetros, observar representaciones visuales y contrastar resultados numéricos en los siguientes temas:

1. Campo y potencial de cargas puntuales, incluida la trayectoria de una partícula.
2. Capacitor de placas paralelas con vacío o dieléctrico.
3. Circuitos resistivos en serie, paralelo y configuración mixta, además de ley de Ohm.
4. Fuentes magnéticas idealizadas y movimiento de una carga bajo la fuerza de Lorentz.

No se modelan dispositivos no lineales reales, efectos térmicos, pérdidas dieléctricas, saturación magnética, radiación ni campos retardados.

# Fundamentos físicos implementados

## Electrostática

El campo de una carga puntual se obtiene con la ley de Coulomb y se combina por superposición:

```
E(r) = k * sum(q_i * (r - r_i) / |r - r_i|^3)
```

El potencial se calcula como una suma escalar:

```
V(r) = k * sum(q_i / |r - r_i|)
```

La constante utilizada es `k = 8.9875517923 x 10^9 N m^2/C^2`. Las coordenadas de la interfaz se expresan en centímetros y se convierten a metros antes del cálculo. El backend ignora contribuciones a menos de un micrómetro del centro de una carga para evitar la singularidad.

La trayectoria electrostática utiliza `a = qE/m` y resuelve el sistema acoplado de posición y velocidad con Runge-Kutta de cuarto orden. Se limita el número de pasos a 5000 y la simulación se detiene si aparecen valores no finitos o magnitudes fuera de los límites de seguridad numérica.

## Capacitores y dieléctricos

El capacitor de placas paralelas usa:

- `C = epsilon_0 * epsilon_r * A / d`
- `Q = C * V`
- `E = V / d`
- `U = 1/2 * C * V^2`
- `u = 1/2 * epsilon * E^2`
- `P = epsilon_0 * (epsilon_r - 1) * E`

Se emplea `epsilon_0 = 8.8541878128 x 10^-12 F/m`. Con la batería conectada se conserva el voltaje; al desconectarla se conserva la carga aislada y el voltaje se recalcula como `V = Q/C`. El modelo supone placas paralelas ideales, dieléctrico lineal, homogéneo e isotrópico y ausencia de efectos de borde.

## Circuitos eléctricos

La relación central es `V = I * R`. Para resistencias en serie se suman los valores. Para ramas en paralelo se utiliza:

```
1/R_eq = sum(1/R_i)
```

La potencia se calcula con formas equivalentes: `P = V * I`, `P = I^2 * R` o `P = V^2/R`. La configuración mixta implementada coloca el primer bloque resistivo en serie con el paralelo de los restantes. El constructor de circuitos amplía este modelo con componentes en serie, ramas paralelas, focos resistivos e interruptores ideales.

Un interruptor abierto o una rama vacía se trata como camino no conductor. El backend Rust admite como máximo veinte resistencias positivas y exige voltajes finitos no negativos.

## Magnetismo y fuerza de Lorentz

La fuerza total se calcula como:

```
F = q * (E + v x B)
```

El backend devuelve por separado las contribuciones eléctrica y magnética, la fuerza total, su magnitud y la aceleración `a = F/m`.

Las fuentes de corriente usan modelos ideales:

- Conductor rectilíneo infinito: `B = mu_0 * I / (2*pi*r)`.
- Centro de una espira: `B = mu_0 * N * I / (2*R)`.
- Solenoide largo: `B = mu_0 * N * I / L`.

La trayectoria bidimensional bajo campos uniformes se integra con Runge-Kutta de cuarto orden, con entre 80 y 1800 pasos. También se calculan el periodo ciclotrón `T = 2*pi*m/|qB|` y la deriva ideal de campos cruzados.

# Arquitectura del software

| Capa | Tecnología | Responsabilidad |
|---|---|---|
| Interfaz | React 19 y TypeScript | Estado, controles, navegación, accesibilidad y presentación. |
| Visualización | SVG y CSS | Diagramas, escenas 2D/3D proyectadas, líneas, trayectorias y animaciones. |
| Adaptador | `src/services/physicsBackend.ts` | Detecta Tauri, normaliza contratos y selecciona backend Rust o respaldo TypeScript. |
| Modelos web | `src/physics` | Cálculos locales para desarrollo en navegador y simulaciones visuales. |
| Backend | Rust y comandos Tauri | Validación de entradas y cálculos físicos verificables. |
| Empaquetado | Tauri 2 y Vite | Aplicación de escritorio y compilación del frontend. |

### Flujo de datos

1. El usuario cambia un parámetro en una estación React.
2. El componente actualiza el estado y prepara valores en unidades SI.
3. El adaptador invoca un comando Tauri cuando existe el runtime de escritorio.
4. En modo navegador, el adaptador ejecuta la implementación TypeScript equivalente.
5. El resultado tipado regresa a la interfaz y actualiza paneles y visualizaciones.

Los comandos Tauri expuestos son `electric_field_at`, `electric_potential_at`, `parallel_plate_capacitor`, `capacitor_energy`, `solve_resistor_circuit`, `solve_ohms_law`, `magnetic_field_from_current` y `lorentz_force`.

# Algoritmos y visualización

### Líneas y superficies eléctricas

Las líneas de campo se generan desde semillas alrededor de las cargas y avanzan en pasos normalizados según la dirección local del campo. El recorrido termina al acercarse a una carga o alcanzar el límite de iteraciones.

Las equipotenciales 2D se obtienen mediante muestreo regular de potencial y detección interpolada de cruces por nivel en cada celda, equivalente a un esquema de marching squares. La vista 3D muestrea una cuadrícula volumétrica, divide cada cubo en tetraedros, interpola intersecciones con niveles de potencial y triangula polígonos aproximados.

### Integración temporal

Las trayectorias electrostática y magnética usan Runge-Kutta de cuarto orden. En cada paso se evalúan cuatro pendientes para posición y velocidad, y se combina su promedio ponderado. Este método mejora la estabilidad y precisión frente a Euler para un costo aceptable en una aplicación interactiva.

### Circuitos construidos

El solver clasifica componentes en un tramo en serie y ramas paralelas. Calcula resistencia por camino, elimina ramas abiertas de la conductancia, obtiene la resistencia equivalente y distribuye corrientes y voltajes. Los resultados de cada componente se derivan de su corriente y resistencia.

### Cartografía magnética

La cartografía usa un modelo vectorial relativo para imán o electroimán, integra líneas de flujo desde semillas y desplaza una sonda interactiva. La lectura representa intensidad y dirección relativas; los comandos Rust proporcionan magnitudes absolutas para geometrías ideales de corriente.

### Proyección y cámara

Las vistas 3D son proyecciones matemáticas sobre SVG. La cámara aplica rotaciones, profundidad, escala, paneo y zoom. La geometría se ordena por profundidad cuando es necesario. Esta técnica evita una dependencia de motor 3D y mantiene el proyecto liviano, aunque no constituye una solución de renderizado físico.

## Validación de entradas y seguridad numérica

- El backend rechaza valores no finitos.
- Las distancias, áreas, masas, radios, longitudes y resistencias físicas deben ser positivas.
- Electrostática admite hasta diez cargas puntuales.
- Circuitos admite hasta veinte resistencias.
- El modelo mixto requiere una resistencia en serie y al menos dos en paralelo.
- Las simulaciones temporales limitan pasos y terminan ante resultados no finitos.
- Los nombres de campos de respuesta incluyen unidades para evitar conversiones implícitas.

## Pruebas y validación

### Verificaciones ejecutadas

| Verificación | Resultado |
| --- | --- |
| `npm run build` | Aprobada. TypeScript y Vite generaron la compilación de producción. |

| Verificación | Resultado |
|---|---|
| `cargo test --manifest-path src-tauri/Cargo.toml` | Aprobada. 15 pruebas pasaron, 0 fallaron. |
| Revisión visual de Inicio | Aprobada. Las cuatro estaciones son accesibles. |
| Campo eléctrico | Aprobada. Casos, vistas, controles de carga y resultados en P visibles. |
| Capacitores | Aprobada. Geometría, medio, batería y magnitudes calculadas visibles. |
| Circuitos | Aprobada. Plantillas, constructor, fuente y resultados visibles. |
| Magnetismo | Aprobada. Cartografía, sonda, fuente y acceso a cámara de Lorentz visibles. |

### Cobertura unitaria en Rust

Las quince pruebas verifican:

- Campo y potencial de una carga puntual a un metro.
- Rechazo de más de diez cargas.
- Capacitancia en vacío, aumento por dieléctrico, energía y rechazo de distancia cero.
- Circuitos serie, paralelo y mixto, además de ley de Ohm.
- Fuerza de Lorentz, combinación de fuerzas, campo de conductor recto y campo de solenoide.

## Limitaciones y riesgos conocidos

1. Las visualizaciones 3D y las líneas de campo son aproximaciones discretas; no deben interpretarse como una solución de elementos finitos.
2. Las fuentes magnéticas de la cartografía son modelos cualitativos distintos de una geometría industrial completa.
3. El modo navegador utiliza cálculos TypeScript equivalentes, pero la validación más estricta reside en Rust.
4. No existe una suite automatizada de interacción del frontend; la validación actual combina compilación, pruebas del backend y revisión visual.
5. Las fórmulas presuponen condiciones ideales y no incorporan tolerancias, temperatura ni pérdidas.

## Conclusiones

NexoVolt cumple el objetivo de ofrecer un laboratorio educativo coherente para cuatro unidades de electromagnetismo. La separación entre interfaz, adaptador y backend reduce el acoplamiento; los contratos con unidades explícitas facilitan la verificación; y las pruebas unitarias cubren los cálculos centrales. La compilación y las quince pruebas aprobadas respaldan la entrega actual.